Práctica N°6: Uso de Paquetes

Índice

1. Objetivos **2**
- 1.1. Objetivo general . . . 2
- 1.2. Objetivos específicos . . . 2

2. Configurar VHDL-2008 en Vivado **2**
- 2.1. Forzar VHDL-2008 por archivo (si algún fichero no hereda) . . . 3

3. Marco teórico **4**
- 3.1. Paquetes en VHDL y su utilidad . . . 4

4. Descripción del código **5**
- 4.1. Paquete `alu_types_pkg` . . . 5
- 4.2. Paquete `alu_ops_pkg` . . . 8
  - 4.2.1. Suma: `alu_add_unsigned4` . . . 13
  - 4.2.2. Resta: `alu_subtract_unsigned4` . . . 14
  - 4.2.3. Multiplicación: `alu_multiply_unsigned4` . . . 14
  - 4.2.4. División: `alu_divide_unsigned4` . . . 14
- 4.3. Paquete `sevenseg_pkg` . . . 15
- 4.4. Módulo `sevenseg_mux` (8 dígitos) . . . 19
- 4.5. `alu_top`: integración y visualización . . . 23

5. Asignación de pines (XDC) **29**

6. Resultados **31**

1. Objetivos

1.1. Objetivo general

Diseñar, implementar y verificar una **ALU de 4 bits** en **VHDL-2008** que utilice **paquetes** para (i) los tipos y operaciones aritméticas (*add, subtract, multiply, divide*) y (ii) la **presentación en displays de 7 segmentos**, integrando todo en un único módulo *top* capaz de mostrar **A** (en dos dígitos), **B** (en dos dígitos) y el **resultado** (en cuatro dígitos), con manejo explícito de *flags* y del caso *división por cero*, sintetizable para **Nexys A7-100T** en **Vivado 2023.2**.

1.2. Objetivos específicos

- Definir en un paquete de tipos una *API* clara: subtipos sin signo de 4 y 8 bits, el tipo enumerado de operación y un *record* de banderas (*carry, borrow, zero, overflow, divide_by_zero*) junto al *record* de resultado de la ALU.
- Implementar en un paquete de operaciones funciones puras y combinacionales para **suma**, **resta**, **multiplicación** (8 bits) y **división** (cociente y residuo), con manejo explícito de **división por cero**; emplear exclusivamente `ieee.numeric_std`.
- Diseñar un paquete de visualización para 7 segmentos: LUT `HEX`→`7SEG` (activos en bajo) y **conversores decimales** (4 bits → 2 dígitos, 8 bits → 4 dígitos) para presentar A, B y el resultado en formato decimal.
- Desarrollar un **driver de multiplexado de 8 dígitos** (AN7..AN0) con frecuencia de refresco por dígito $\geq$ 1 kHz para evitar parpadeo, y puntos decimales como indicadores de error en *divide by zero*.
- Integrar en un *top* único: mapeo A→AN7..AN6, B→AN5..AN4, resultado →AN3..AN0; exportar *flags* a LEDs; preparar **XDC** con IOSTANDARD correcto.

2. Configurar VHDL-2008 en Vivado

Para evitar errores de síntesis como `OTHERS in array aggregate without constraining context` y asegurar compatibilidad con paquetes y `numeric_std`, es indispensable fijar el estándar **VHDL-2008** tanto a nivel de proyecto como (si hace falta) por archivo.

2.1. Forzar VHDL-2008 por archivo (si algún fichero no hereda)

1. En el **Sources**, clic derecho sobre el archivo `.vhd` problemático → **Source File Properties**.
2. En **VHDL Language Version**, seleccione **VHDL 2008**.
3. Aplique cambios y vuelva a sintetizar.

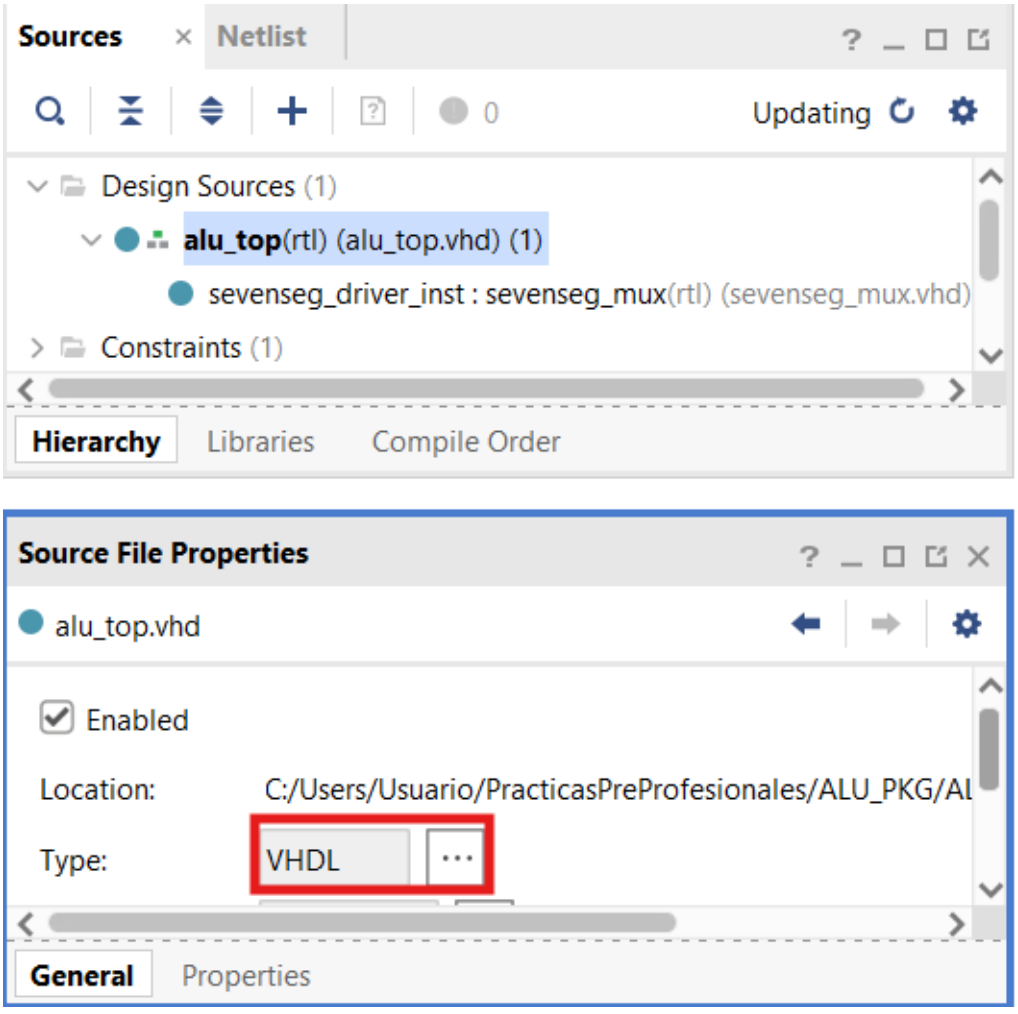


Figura 1: Propiedades por archivo: forzar *VHDL 2008* en un `.vhd` concreto.

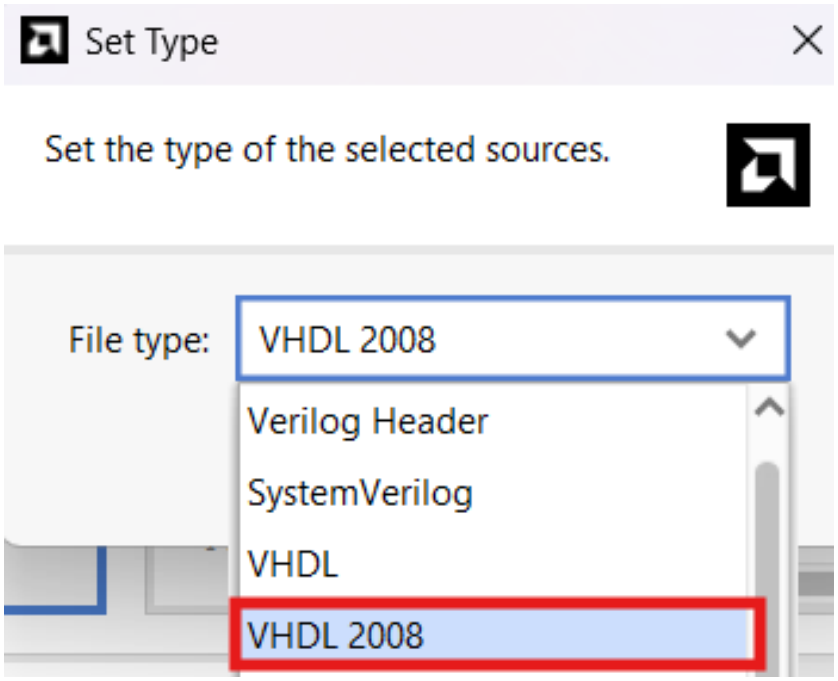


Figura 2: Propiedades por archivo: forzar *VHDL 2008* en un `.vhd` concreto paso 2.

3. Marco teórico

3.1. Paquetes en VHDL y su utilidad

En VHDL, un **paquete** (*package*) es una unidad de diseño que agrupa *declaraciones reutilizables*: tipos (`type`, `subtype`), constantes, funciones y procedimientos. Un paquete se compone de una **declaración** (`package`)—donde se exponen las interfaces—y, opcionalmente, un **cuerpo** (`package body`)—donde se implementan las subrutinas. Otros módulos obtienen visibilidad usando `library`/`use` (p. ej., `use work.mi_pkg.all;`).

¿Para qué sirven?

- **Modularidad y reutilización**: centralizan tipos, constantes (p. ej., LUTs) y funciones comunes (p. ej., operaciones de ALU, conversores a 7 segmentos).
- **Interfaces estables**: evitan duplicación y desalineación de definiciones entre el *top*, testbenches y bloques internos.
- **Mantenibilidad y portabilidad**: cambiar una definición en el paquete actualiza a todos los consumidores.

- **Buenas prácticas de síntesis**: funciones *puras*, constantes bien tipadas y **sin señales ni procesos** en el paquete (solo declaraciones), favoreciendo resultados predecibles en hardware.
- **Parametrización** (VHDL-2008): permiten *package generics* para ajustar valores globales sin tocar múltiples archivos.

4. Descripción del código

4.1. Paquete `alu_types_pkg`

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  -- ========================
6  -- Paquete de tipos y constantes para la ALU didactica
       de 4 bits
7  -- Define subtipos con nombres explicitos, el conjunto
       de operaciones,
8  -- un registro de banderas (flags) y el tipo de
       resultado de la ALU.
9  -- ========================
10 package alu_types_pkg is
11   -- Tipos base con nombres claros
12   subtype unsigned_4bit_t is unsigned(3 downto 0);
13   subtype unsigned_8bit_t is unsigned(7 downto 0);
14
15   -- Conjunto explicito de operaciones
16   type operation_t is (OPERATION_ADD, OPERATION_SUBTRACT
       , OPERATION_MULTIPLY, OPERATION_DIVIDE);
17
18   -- Banderas de estado con nombres completos
19   type flags_record_t is record
20     carry_flag            : std_logic;  -- valido para
         suma sin signo
21     borrow_flag           : std_logic;  -- valido para
         resta sin signo (1 = hubo prestamo)
22     zero_flag             : std_logic;  -- 1 cuando el
         resultado relevante es cero
23     overflow_flag         : std_logic;  -- p/ unsigned:
         ADD=carry; SUB=borrow; MUL=parte alta != 0
```

```
24      divide_by_zero_flag    : std_logic;  -- 1 si B = 0 en
            division
25    end record;
26
27    -- Resultado de la ALU empaquetado y autoexplicativo
28    type alu_result_record_t is record
29      result_8bit_unsigned  : unsigned_8bit_t;    --
            resultado principal de 8 bits (ADD/SUB/MUL).
30      quotient_4bit_unsigned  : unsigned_4bit_t; --
            cociente (DIV)
31      remainder_4bit_unsigned : unsigned_4bit_t; --
            residuo  (DIV)
32      flags                   : flags_record_t;  --
            banderas
33    end record;
34  end package;
35
36  package body alu_types_pkg is
37  end package body;
```

Listing 1: Paquete de tipos y constantes para la ALU didáctica.

Qué resuelve este paquete y por qué existe. Este paquete **aclara contratos** entre módulos: define *qué* tamaños de datos se usarán, *qué* operaciones reconoce la ALU y *cómo* se reportan los resultados y estados (banderas). Al centralizar estos tipos: (i) evitamos inconsistencias de ancho, (ii) documentamos la interfaz, y (iii) facilitamos pruebas y mantenimiento.

Importaciones (`library/use`). Se usan `std_logic_1164` para tipos lógicos y `numeric_std` para aritmética **sin signo** y **con signo**. Elegimos `numeric_std` (en lugar de paquetes no estándar como `std_logic_unsigned`) para mantener portabilidad y un comportamiento bien definido en síntesis.

Subtipos con semántica explícita.

- `unsigned_4bit_t` → operandos de la ALU (A, B), rango 0.,15.
- `unsigned_8bit_t` → resultados que pueden exceder 4 bits (p. ej., $15 \times 15 = 225$), rango 0.,255.

Usar *subtypes* en lugar de `unsigned(3 downto 0)` "a pelo" obliga a una **consistencia de tamaños** y mejora la legibilidad.

Tipo enumerado de operación `operation_t`**.** Define el **dominio de operaciones** de la práctica: `ADD`, `SUBTRACT`, `MULTIPLY`, `DIVIDE`. Ventaja: el `top` y los testbenches pueden *seleccionar* la operación con nombres semánticos en lugar de literales numéricos, reduciendo errores.

Banderas de estado `flags_record_t`**.** Las banderas capturan **condiciones relevantes** por operación. Úsalas como telemetría y para evaluación:

Bandera	**Tipo**	**Se activa cuando**	**Aplica en**
`carry_flag`	`std_logic`	Suma excede 4 bits	ADD
`borrow_flag`	`std_logic`	$A < B$ en resta	SUBTRACT
`zero_flag`	`std_logic`	Resultado relevante es 0	Todas
`overflow_flag`	`std_logic`	Convención unsigned	ADD/SUB/MUL
`divide_by_zero_flag`	`std_logic`	Divisor $B = 0$	DIVIDE

Registro de resultado `alu_result_record_t`**.** Estructura canónica que **agrega todo lo que un consumidor necesita**:

1. `result_8bit_unsigned`: canal principal para ADD, SUB y MUL; permite visualizar tanto la parte baja como la alta (si existe).
2. `quotient_4bit_unsigned` y `remainder_4bit_unsigned`: salidas claras para DIV (evita ambigüedades).
3. `flags`: el *estado* de la operación viaja junto al dato, simplificando verificación y depuración.

Para DIV, además se especifica que `result_8bit_unsigned` se empaqueta como [*remainder*][*quotient*] (nibble alto y bajo). Esto facilita inspección rápida en simulación/ILA sin tener que mirar múltiples señales.

4.2. Paquete alu_ops_pkg

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  use work.alu_types_pkg.all;
6
7  -- ================================
8  -- Paquete de operaciones puras de la ALU (
       combinacionales, didacticas).
9  -- Todas trabajan en UNSIGNED (sin signo) para mantener
       la practica simple.
10 -- Se evita "(others => ...)" donde pueda faltar
       contexto, usando to_unsigned.
11 -- ================================
12 package alu_ops_pkg is
13   function alu_add_unsigned4
14   (
15     operand_a_unsigned_4bit : unsigned_4bit_t;
16     operand_b_unsigned_4bit : unsigned_4bit_t
17   ) return alu_result_record_t;
18
19   function alu_subtract_unsigned4
20   (
21     operand_a_unsigned_4bit : unsigned_4bit_t;
22     operand_b_unsigned_4bit : unsigned_4bit_t
23   ) return alu_result_record_t;
24
25   function alu_multiply_unsigned4
26   (
27     operand_a_unsigned_4bit : unsigned_4bit_t;
28     operand_b_unsigned_4bit : unsigned_4bit_t
29   ) return alu_result_record_t;
30
31   function alu_divide_unsigned4
32   (
33     operand_a_unsigned_4bit : unsigned_4bit_t;
34     operand_b_unsigned_4bit : unsigned_4bit_t
35   ) return alu_result_record_t;  -- maneja division por
       cero
36 end package;
37
```

```
38 package body alu_ops_pkg is
39   -- Inicializador de banderas a cero (claridad)
40   function blank_flags return flags_record_t is
41     variable flags_local : flags_record_t;
42   begin
43     flags_local.carry_flag           := '0';
44     flags_local.borrow_flag          := '0';
45     flags_local.zero_flag            := '0';
46     flags_local.overflow_flag        := '0';
47     flags_local.divide_by_zero_flag  := '0';
48     return flags_local;
49   end function;
50
51   -- -------------------------- S U M A
        ---------------------------------------
52   function alu_add_unsigned4
53   (
54     operand_a_unsigned_4bit : unsigned_4bit_t;
55     operand_b_unsigned_4bit : unsigned_4bit_t
56   ) return alu_result_record_t is
57     variable alu_result_local           :
          alu_result_record_t;
58     variable flags_local                :
          flags_record_t := blank_flags;
59     variable extended_sum_5bit_unsigned  : unsigned(4
          downto 0);
60   begin
61     extended_sum_5bit_unsigned := ('0' &
          operand_a_unsigned_4bit) + ('0' &
          operand_b_unsigned_4bit);
62
63     -- Empaquetamos en 8 bits: parte alta incluye el
          carry para visualizarlo.
64     alu_result_local.result_8bit_unsigned := ("000" &
          extended_sum_5bit_unsigned(4)) &
          extended_sum_5bit_unsigned(3 downto 0);
65
66     -- Valores no usados en esta operacion a cero por
          claridad (ancho explicito)
67     alu_result_local.quotient_4bit_unsigned  :=
          to_unsigned(0, 4);
68     alu_result_local.remainder_4bit_unsigned :=
          to_unsigned(0, 4);
```

```
69
70      -- Banderas
71      flags_local.carry_flag    :=
           extended_sum_5bit_unsigned(4);
72      flags_local.zero_flag     := '1' when
           alu_result_local.result_8bit_unsigned =
           to_unsigned(0, 8) else '0';
73      flags_local.overflow_flag := flags_local.carry_flag;
             -- convencion didactica para UNSIGNED
74
75      alu_result_local.flags := flags_local;
76      return alu_result_local;
77    end function;
78
79    -- -------------------------- R E S T A
         -------------------------------------
80    function alu_subtract_unsigned4
81    (
82      operand_a_unsigned_4bit : unsigned_4bit_t;
83      operand_b_unsigned_4bit : unsigned_4bit_t
84    ) return alu_result_record_t is
85      variable alu_result_local                :
           alu_result_record_t;
86      variable flags_local                     :
           flags_record_t := blank_flags;
87      variable diff_signed_5bit                : signed(4
            downto 0);
88      variable operand_a_unsigned_4bit_copy    :
           unsigned_4bit_t;
89      variable operand_b_unsigned_4bit_copy    :
           unsigned_4bit_t;
90    begin
91      operand_a_unsigned_4bit_copy :=
           operand_a_unsigned_4bit;
92      operand_b_unsigned_4bit_copy :=
           operand_b_unsigned_4bit;
93
94      -- Extension a 5 bits y operacion en signed para
           captar signo del resultado
95      diff_signed_5bit := signed('0' & std_logic_vector(
           operand_a_unsigned_4bit_copy))
96                        - signed('0' & std_logic_vector(
                             operand_b_unsigned_4bit_copy));
```

```
97
98      -- Resultado en los 4 bits bajos, resto en 0 por
            claridad
99      alu_result_local.result_8bit_unsigned             :=
            to_unsigned(0, 8);
100     alu_result_local.result_8bit_unsigned(3 downto 0) :=
            unsigned(std_logic_vector(diff_signed_5bit(3
           downto 0)));
101
102     -- No aplica cociente ni residuo
103     alu_result_local.quotient_4bit_unsigned  :=
           to_unsigned(0, 4);
104     alu_result_local.remainder_4bit_unsigned :=
           to_unsigned(0, 4);
105
106     -- Banderas: borrow cuando A < B en UNSIGNED
107     flags_local.borrow_flag   := '1' when
           operand_a_unsigned_4bit_copy <
           operand_b_unsigned_4bit_copy else '0';
108     flags_local.zero_flag     := '1' when (
           alu_result_local.result_8bit_unsigned(3 downto 0)
            = "0000" and flags_local.borrow_flag = '0') else
            '0';
109     flags_local.overflow_flag := flags_local.borrow_flag
           ;  -- convencion didactica para UNSIGNED
110
111     alu_result_local.flags := flags_local;
112     return alu_result_local;
113   end function;
114
115   -- --------------------- M U L T I P L I C A C I O N
          -------------------------
116   function alu_multiply_unsigned4
117   (
118     operand_a_unsigned_4bit : unsigned_4bit_t;
119     operand_b_unsigned_4bit : unsigned_4bit_t
120   ) return alu_result_record_t is
121     variable alu_result_local         :
           alu_result_record_t;
122     variable flags_local              : flags_record_t
           := blank_flags;
123     variable product_unsigned_8bit    : unsigned_8bit_t;
124   begin
```

```
125     product_unsigned_8bit := unsigned(
           operand_a_unsigned_4bit) * unsigned(
           operand_b_unsigned_4bit);
126
127     alu_result_local.result_8bit_unsigned    :=
           product_unsigned_8bit;
128     alu_result_local.quotient_4bit_unsigned  :=
           to_unsigned(0, 4);
129     alu_result_local.remainder_4bit_unsigned :=
           to_unsigned(0, 4);
130
131     flags_local.zero_flag     := '1' when
           product_unsigned_8bit = to_unsigned(0, 8) else
           '0';
132     flags_local.overflow_flag := '1' when
           product_unsigned_8bit(7 downto 4) /= "0000" else
           '0';
133
134     alu_result_local.flags := flags_local;
135     return alu_result_local;
136   end function;
137
138   -- ------------------------ D I V I S I o N
         ---------------------------------
139   function alu_divide_unsigned4
140   (
141     operand_a_unsigned_4bit : unsigned_4bit_t;
142     operand_b_unsigned_4bit : unsigned_4bit_t
143   ) return alu_result_record_t is
144     variable alu_result_local         :
           alu_result_record_t;
145     variable flags_local              : flags_record_t
           := blank_flags;
146     variable quotient_unsigned_4bit   : unsigned_4bit_t;
147     variable remainder_unsigned_4bit  : unsigned_4bit_t;
148   begin
149     if operand_b_unsigned_4bit = to_unsigned(0, 4) then
150       -- Division por cero: bandera y convencion
             didactica (quot=0, rem=A)
151       flags_local.divide_by_zero_flag := '1';
152       quotient_unsigned_4bit  := to_unsigned(0, 4);
153       remainder_unsigned_4bit := operand_a_unsigned_4bit
             ;
```

```
154     else
155       quotient_unsigned_4bit  := operand_a_unsigned_4bit
              / operand_b_unsigned_4bit;
156       remainder_unsigned_4bit := operand_a_unsigned_4bit
              mod operand_b_unsigned_4bit;
157     end if;
158
159     alu_result_local.quotient_4bit_unsigned  :=
           quotient_unsigned_4bit;
160     alu_result_local.remainder_4bit_unsigned :=
           remainder_unsigned_4bit;
161     alu_result_local.result_8bit_unsigned    :=
           remainder_unsigned_4bit & quotient_unsigned_4bit;
            -- [rem][quot]
162
163     flags_local.zero_flag := '1'
164       when (quotient_unsigned_4bit  = to_unsigned(0, 4)
              and
165                remainder_unsigned_4bit = to_unsigned(0, 4))
166       else '0';
167
168     alu_result_local.flags := flags_local;
169     return alu_result_local;
170   end function;
171 end package body;
```

Listing 2: Operaciones combinacionales de la ALU (unsigned).

Visión general. Este paquete implementa las **cuatro operaciones aritméticas** sobre operandos de 4 bits *sin signo*, devolviendo siempre un `alu_result_record_t` con datos y banderas. Todo es **combinacional** y se apoya en `numeric_std`. La función auxiliar `blank_flags` garantiza que cada bandera inicia en cero.

4.2.1. Suma: `alu_add_unsigned4`

Propósito. Sumar $A + B$ con extensión a 5 bits para capturar el *carry* y empaquetar el resultado en 8 bits. **Entradas/Salidas.** Dos `unsigned_4bit_t` y un `alu_result_record_t`. **Flujo.**

1. Se concatena un `'0'` a cada operando: `('0' & A)` y `('0' & B)` $\Rightarrow$ suma de 5 bits.
2. **Resultado**: se forma `result_8bit_unsigned` como "000"&$carry$&$sum(3.,0)$ para visualizar el *carry* en el nibble alto.

3. **Banderas**: `carry_flag := bit4`, `zero_flag` si el resultado de 8 bits es 0, y `overflow_flag := carry_flag` (convención para unsigned).

Observación didáctica. Mantener el *carry* visible en `result_8bit_unsigned` permite al estudiante corroborar el LED de `carry` y lo que "ve" en los 7 segmentos.

4.2.2. Resta: `alu_subtract_unsigned4`

Propósito. Calcular $A-B$ en *unsigned*. **Entradas/Salidas.** Dos `unsigned_4bit_t` y un `alu_result_record_t`. **Flujo.**

1. Se extiende a 5 bits y se opera en `signed` para capturar el signo intermedio sin ambigüedad: `signed('0' & A)` − `signed('0' & B)`.

2. **Resultado**: se toman los 4 bits bajos del *diff* y se colocan en result_8bit_unsigned(3..0) (el resto del byte se pone a cero).

3. **Banderas**: `borrow_flag := (A <B)`, `zero_flag` si el nibble de resultado es `0000` y `borrow=0`, y `overflow_flag := borrow_flag` como convención para *unsigned*.

4.2.3. Multiplicación: `alu_multiply_unsigned4`

Propósito. Calcular $A \times B$ y devolver un resultado de 8 bits. **Entradas/Salidas.** Dos `unsigned_4bit_t` y un `alu_result_record_t`. **Flujo.**

1. Se realiza la multiplicación natural en `unsigned`, produciendo un `unsigned_8bit_t`.

2. **Resultado**: `result_8bit_unsigned := producto(7..0)`.

3. **Banderas**: `zero_flag` si el producto es 0; `overflow_flag` si la parte alta (7.,4) es distinta de `0000` (indicando que no cabe en 4 bits).

Comentario. Para $15 \times 15 = 225$, el estudiante debe observar `overflow_flag=1` y el decimal `0225` en displays.

4.2.4. División: `alu_divide_unsigned4`

Propósito. Calcular el **cociente** y el **residuo** de A/B, manejando explícitamente la división por cero. **Entradas/Salidas.** Dos `unsigned_4bit_t` y un `alu_result_record_t`. **Flujo.**

1. Si $B = 0$: `divide_by_zero_flag := '1'`, `quotient := 0`, `remainder := A`.

2. Si $B \neq 0$: `quotient := A / B`, `remainder := A mod B`.

3. **Resultado**: se empaqueta además en `result_8bit_unsigned := remainder & quotient` ($[rem][quot]$) para inspección rápida.

4. **Banderas**: `zero_flag` si $quotient = 0$ y $remainder = 0$ (i.e., $A = 0$).

4.3. Paquete `sevenseg_pkg`

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- ================================
-- Paquete de utilidades para displays de 7 segmentos (activos en bajo).
-- Proporciona:
--   - LUT HEX->7SEG
--   - Conversion nibble (0..15) -> dos digitos decimales (decenas, unidades)
--   - Conversion byte  (0..255) -> cuatro digitos decimales (mil, cen, dec, uni)
-- Orden de segmentos: seg(6 downto 0) = CA..CG, activo en bajo.
-- ================================
package sevenseg_pkg is
  type seven_segment_lut_t is array (natural range <>) of std_logic_vector(6 downto 0);

  -- Tabla para 0..F, activos en bajo
  constant HEX_TO_SEVEN_SEGMENT_ACTIVE_LOW : seven_segment_lut_t(0 to 15) := (
    -- 0       1       2       3       4       5       6       7
    "0000001","1001111","0010010","0000110","1001100","0100100","0100000","0001111",
    -- 8       9       A       b       C       d       E       F
    "0000000","0000100","0001000","1100000","0110001","1000010","0110000","0111000"
  );

```

```
24    constant SEVEN_SEGMENT_BLANK_ACTIVE_LOW    :
         std_logic_vector(6 downto 0) := (others => '1');
25    constant SEVEN_SEGMENT_CHAR_E_ACTIVE_LOW   :
         std_logic_vector(6 downto 0) := "0110000"; -- 'E'
26
27    -- Conversion directa HEX -> 7seg
28    function unsigned_nibble_to_seven_segment_active_low(
         nibble_unsigned_4bit : unsigned(3 downto 0)) return
          std_logic_vector;
29
30    -- Conversion 0..15 -> (decenas, unidades) en dos
         nibbles [tens(7..4) & ones(3..0)]
31    function u4_to_two_decimal_nibbles(value_u4 : unsigned
         (3 downto 0)) return unsigned;
32
33    -- Conversion 0..255 -> (miles, centenas, decenas,
         unidades) en cuatro nibbles [M C D U]
34    function u8_to_four_decimal_nibbles(value_u8 :
         unsigned(7 downto 0)) return unsigned;
35  end package;
36
37  package body sevenseg_pkg is
38    function unsigned_nibble_to_seven_segment_active_low
39    (
40      nibble_unsigned_4bit : unsigned(3 downto 0)
41    ) return std_logic_vector is
42    begin
43      return HEX_TO_SEVEN_SEGMENT_ACTIVE_LOW(to_integer(
           nibble_unsigned_4bit));
44    end function;
45
46    -- 0..15 -> 2 digitos decimales (00..15)
47    function u4_to_two_decimal_nibbles(value_u4 : unsigned
         (3 downto 0)) return unsigned is
48      variable value_nat  : natural := to_integer(value_u4
           );  -- 0..15
49      variable tens_nat   : natural;
50      variable ones_nat   : natural;
51      variable result_u8  : unsigned(7 downto 0);
52    begin
53      tens_nat := value_nat / 10;  -- 0..1
54      ones_nat := value_nat mod 10;-- 0..9
```

```
55      result_u8 := to_unsigned(tens_nat, 4) & to_unsigned(
            ones_nat, 4);
56      return result_u8;
57    end function;
58
59    -- 0..255 -> 4 digitos decimales (0000..0255)
60    function u8_to_four_decimal_nibbles(value_u8 :
          unsigned(7 downto 0)) return unsigned is
61      variable value_nat   : natural := to_integer(
            value_u8); -- 0..255
62      variable thousands   : natural;
63      variable hundreds    : natural;
64      variable tens        : natural;
65      variable ones        : natural;
66      variable result_u16  : unsigned(15 downto 0);
67    begin
68      thousands := (value_nat / 1000) mod 10; -- 0
69      hundreds  := (value_nat / 100)  mod 10; -- 0..2
70      tens      := (value_nat / 10)   mod 10; -- 0..9
71      ones      :=  value_nat          mod 10;-- 0..9
72      result_u16 := to_unsigned(thousands, 4) &
73                    to_unsigned(hundreds , 4) &
74                    to_unsigned(tens     , 4) &
75                    to_unsigned(ones     , 4);
76      return result_u16;
77    end function;
78  end package body;
```

Listing 3: Utilidades para displays de 7 segmentos (activos en bajo).

Este módulo es el "intérprete" entre números y los siete segmentos. Tú le das un valor y él te devuelve el patrón exacto de segmentos (CA..CG) que hay que encender. Ojo: los segmentos y ánodos son **activos en bajo**, así que un `'0'` enciende y un `'1'` apaga.

Un nibble es un grupo de 4 bits —la "mitad" de un byte— capaz de representar valores enteros 0..15 en unsigned. Por eso coincide exactamente con un dígito hexadecimal (0–F) y es muy práctico en hardware digital: muchas ALUs básicas operan sobre nibbles, y los decodificadores a 7 segmentos convierten justo ese rango a patrones de segmentos. En resumen: cuando piensas en "un dígito hex", estás pensando en un nibble.

LUT `HEX_TO_SEVEN_SEGMENT_ACTIVE_LOW`**.** Es la "chuleta" de 16 entradas (0..F) que mapea cada nibble a su patrón de 7 segmentos. El orden acordado

es `seg(6...0) = CA..CG`. Si tu placa usa otro orden, el problema *no* está en la ALU: solo cambias esta tabla y listo.

Funciones listas para usar.

- `unsigned_nibble_to_seven_segment_active_low(n)`: la más directa. Entra un nibble (0..15) y sale el patrón activo en bajo. Ideal para mostrar `A` y `B` en *hex*, pero en esta práctica los mostramos en **decimal**, así que vamos con las siguientes.
- `u4_to_two_decimal_nibbles(value_u4)`: convierte un valor de 4 bits (0..15) a **dos dígitos decimales** (decenas y unidades). Devuelve un `unsigned(7 downto 0)` empacado como [`tens(7..4)`‖`ones(3..0)`]. Con esto, si `A` = 1111 verás **1** y **5** en los dos primeros displays.
- `u8_to_four_decimal_nibbles(value_u8)`: toma un byte (0..255) y te entrega **cuatro dígitos decimales** (miles, centenas, decenas, unidades) empaquetados en 16 bits, [`M C D U`]. Para nuestra ALU, el dígito de miles será 0 salvo casos futuros (extensiones a más bits).

Constantes de utilidad. `SEVEN_SEGMENT_BLANK_ACTIVE_LOW` apaga todo (muy útil para "borrar" dígitos sin ruido visual). `SEVEN_SEGMENT_CHAR_E_ACTIVE_LOW` pinta una "E" mayúscula; la usamos para marcar *división por cero* de forma inequívoca.

Por qué decimal aquí (y no hex directo). En hardware es habitual mostrar *hex* por pura comodidad. Pero tu requerimiento didáctico es ver `A`, `B` y el *resultado* en **decimal** y por bloques (2 dígitos + 2 dígitos + 4 dígitos). Estas funciones hacen justamente ese puente sin ensuciar el `top` con divisiones y restos.

Detalle técnico que conviene saber. Las conversiones decimales usan divisiones y módulos sobre `natural`. Sí, sintetizan como lógica combinacional; para rangos cortos (0..15 y 0..255) el costo en LUTs es perfectamente razonable a 100 MHz. Si alguna vez escalas a 16 bits, te convendrá un *double-dabble* o un conversor BCD iterativo.

4.4. Módulo sevenseg_mux (8 dígitos)

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  -- ================================
6  -- Mux de 8 displays de 7 segmentos (anodos activos en
       bajo) para Nexys A7.
7  -- Orden de activacion: AN7, AN6, AN5, AN4, AN3, AN2,
       AN1, AN0 (izq a der).
8  -- Entradas: patrones codificados (activos en bajo) por
       digito.
9  -- ================================
10 entity sevenseg_mux is
11   generic(
12     CLOCK_FREQUENCY_HZ        : integer := 100_000_000;
          -- p.ej. 100 MHz
13     DIGIT_REFRESH_RATE_HZ     : integer := 1000
          -- por digito (a 8 kHz total)
14   );
15   port(
16     clk_system                          : in  std_logic;
17     rst_sync_active_high                : in  std_logic;
18
19     digit7_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
20     digit6_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
21     digit5_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
22     digit4_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
23     digit3_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
24     digit2_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
25     digit1_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
26     digit0_segment_pattern_active_low   : in
          std_logic_vector(6 downto 0);
27
```

```
28      decimal_points_active_low_per_digit: in
           std_logic_vector(7 downto 0); -- '0' = encendido
           (act. bajo)
29
30      seg_active_low                       : out
           std_logic_vector(6 downto 0); -- CA..CG
31      dp_active_low                        : out std_logic;
32      an_active_low                        : out
           std_logic_vector(7 downto 0)  -- AN7..AN0
33    );
34  end entity;
35
36  architecture rtl of sevenseg_mux is
37    constant TICKS_PER_DIGIT_STEP : integer :=
         CLOCK_FREQUENCY_HZ / (DIGIT_REFRESH_RATE_HZ * 8);
38
39    signal tick_counter    : integer range 0 to
         TICKS_PER_DIGIT_STEP-1 := 0;
40    signal digit_selector  : unsigned(2 downto 0) := (
         others => '0');
41
42    signal seg_selected_active_low : std_logic_vector(6
         downto 0);
43    signal dp_selected_active_low  : std_logic;
44  begin
45    -- Divisor y selector ciclico de 8 digitos
46    process(clk_system)
47    begin
48      if rising_edge(clk_system) then
49        if rst_sync_active_high = '1' then
50          tick_counter   <= 0;
51          digit_selector <= (others => '0');
52        else
53          if tick_counter = TICKS_PER_DIGIT_STEP-1 then
54            tick_counter   <= 0;
55            digit_selector <= digit_selector + 1;
56          else
57            tick_counter <= tick_counter + 1;
58          end if;
59        end if;
60      end if;
61    end process;
62
```

```
63  -- Seleccion de patron y DP segun digito activo
64  process(digit_selector,
65          digit0_segment_pattern_active_low,
               digit1_segment_pattern_active_low,
66          digit2_segment_pattern_active_low,
               digit3_segment_pattern_active_low,
67          digit4_segment_pattern_active_low,
               digit5_segment_pattern_active_low,
68          digit6_segment_pattern_active_low,
               digit7_segment_pattern_active_low,
69          decimal_points_active_low_per_digit)
70  begin
71    case digit_selector is
72      when "000" => seg_selected_active_low <=
           digit0_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(0);
73      when "001" => seg_selected_active_low <=
           digit1_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(1);
74      when "010" => seg_selected_active_low <=
           digit2_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(2);
75      when "011" => seg_selected_active_low <=
           digit3_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(3);
76      when "100" => seg_selected_active_low <=
           digit4_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(4);
77      when "101" => seg_selected_active_low <=
           digit5_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(5);
78      when "110" => seg_selected_active_low <=
           digit6_segment_pattern_active_low;
           dp_selected_active_low <=
           decimal_points_active_low_per_digit(6);
79      when others=> seg_selected_active_low <=
           digit7_segment_pattern_active_low;
```

```
            dp_selected_active_low <=
            decimal_points_active_low_per_digit(7);
80      end case;
81    end process;
82
83    -- Habilitacion de anodos (activos en bajo): AN7..AN0
84    an_active_low <= "11111110" when digit_selector="000"
        else -- AN0
85                     "11111101" when digit_selector="001"
                          else -- AN1
86                     "11111011" when digit_selector="010"
                          else -- AN2
87                     "11110111" when digit_selector="011"
                          else -- AN3
88                     "11101111" when digit_selector="100"
                          else -- AN4
89                     "11011111" when digit_selector="101"
                          else -- AN5
90                     "10111111" when digit_selector="110"
                          else -- AN6
91                     "01111111";
                                                          --
                          AN7
92
93    seg_active_low <= seg_selected_active_low;
94    dp_active_low  <= dp_selected_active_low;
95  end architecture;
```

Listing 4: Multiplexor para 8 displays de 7 segmentos, activos en bajo (Nexys A7).

¿Qué hace este módulo? Este bloque enciende los **8 dígitos** "a toda velocidad", uno por vez, de forma tan rápida que tus ojos lo perciben como si todos estuvieran encendidos a la vez. Es el clásico *multiplexado*: selecciona un ánodo (activo en bajo), pone el patrón de segmentos para ese dígito, y pasa al siguiente.

Genéricos `CLOCK_FREQUENCY_HZ` y `DIGIT_REFRESH_RATE_HZ`. - **Reloj del sistema** (por defecto 100 MHz) y **tasa de refresco por dígito** (1 kHz) determinan cuántos ciclos dura cada "paso" de dígito. - La constante `TICKS_PER_DIGIT_STEP` se calcula como $\frac{f_{\text{clk}}}{8 \cdot f_{\text{digit}}}$. Con 100 MHz y 1 kHz obtienes $\approx 12{,}5$ k ciclos por dígito, es decir ~ 8 kHz de refresco total: **sin parpadeo**.

4.5. alu_top: integración y visualización

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  use work.alu_types_pkg.all;
6  use work.alu_ops_pkg.all;
7  use work.sevenseg_pkg.all;
8
9  -- ============================
10 -- TOP con 8 displays:
11 --   AN7..AN6 : A en decimal (decenas, unidades)
12 --   AN5..AN4 : B en decimal (decenas, unidades)
13 --   AN3..AN0 : Resultado
14 --       - ADD/SUB/MUL: resultado en decimal (4 digitos)
15 --       - DIV: [quotient(2 digitos)][remainder(2
   digitos)]
16 --       - /0: 'E' en AN0 y resto en blanco (A y B se
   muestran igual)
17 -- ============================
18 entity alu_top is
19   port(
20     clk_system                    : in  std_logic;
                       -- p.ej. 100 MHz
21     rst_sync_active_high          : in  std_logic;
22
23     a_switches_4bit_active_high   : in  std_logic_vector
         (3 downto 0);
24     b_switches_4bit_active_high   : in  std_logic_vector
         (3 downto 0);
25
26     operation_select_2bit         : in  std_logic_vector
         (1 downto 0); -- 00=ADD, 01=SUB, 10=MUL, 11=DIV
27
28     seg_active_low                : out std_logic_vector
         (6 downto 0);
29     dp_active_low                 : out std_logic;
30     an_active_low                 : out std_logic_vector
         (7 downto 0);
31
32     led_carry_flag_active_high          : out std_logic;
33     led_borrow_flag_active_high         : out std_logic;
```

```
34      led_zero_flag_active_high          : out std_logic;
35      led_overflow_flag_active_high      : out std_logic;
36      led_divide_by_zero_flag_active_high: out std_logic
37    );
38  end entity;
39
40  architecture rtl of alu_top is
41    -- Entradas en tipos explicitos
42    signal operand_a_unsigned_4bit : unsigned(3 downto 0);
43    signal operand_b_unsigned_4bit : unsigned(3 downto 0);
44
45    -- Seleccion de operacion
46    signal current_operation : operation_t;
47
48    -- Resultado ALU
49    signal alu_result : alu_result_record_t;
50
51    -- Patrones para los 8 digitos (activos en bajo)
52    signal d7, d6, d5, d4, d3, d2, d1, d0 :
        std_logic_vector(6 downto 0);
53
54    -- DP por digito (activos en bajo): por defecto
        apagados ('1')
55    signal dp_mask_8 : std_logic_vector(7 downto 0) := (
        others => '1');
56  begin
57    -- Casting entradas
58    operand_a_unsigned_4bit <= unsigned(
        a_switches_4bit_active_high);
59    operand_b_unsigned_4bit <= unsigned(
        b_switches_4bit_active_high);
60
61    -- Decodificacion de operacion
62    with operation_select_2bit select current_operation <=
63      OPERATION_ADD      when "00",
64      OPERATION_SUBTRACT when "01",
65      OPERATION_MULTIPLY when "10",
66      OPERATION_DIVIDE   when others;
67
68    -- Nucleo ALU
69    process(operand_a_unsigned_4bit,
        operand_b_unsigned_4bit, current_operation)
70    begin
```

```
71      case current_operation is
72        when OPERATION_ADD      => alu_result <=
             alu_add_unsigned4     (operand_a_unsigned_4bit,
              operand_b_unsigned_4bit);
73        when OPERATION_SUBTRACT => alu_result <=
             alu_subtract_unsigned4(operand_a_unsigned_4bit,
              operand_b_unsigned_4bit);
74        when OPERATION_MULTIPLY => alu_result <=
             alu_multiply_unsigned4(operand_a_unsigned_4bit,
              operand_b_unsigned_4bit);
75        when OPERATION_DIVIDE   => alu_result <=
             alu_divide_unsigned4  (operand_a_unsigned_4bit,
              operand_b_unsigned_4bit);
76      end case;
77    end process;
78
79    -- Formateo para 8 digitos:
80    --  A: AN7..AN6, B: AN5..AN4, Resultado: AN3..AN0
81    process(alu_result, operand_a_unsigned_4bit,
         operand_b_unsigned_4bit, current_operation)
82      variable a_dec2  : unsigned(7 downto 0);   -- [tens
           ][ones]
83      variable b_dec2  : unsigned(7 downto 0);   -- [tens
           ][ones]
84      variable r_dec4  : unsigned(15 downto 0);  -- [th][
           hu][te][on]
85      variable q_dec2  : unsigned(7 downto 0);
86      variable m_dec2  : unsigned(7 downto 0);
87      variable hi4     : unsigned(3 downto 0);
88      variable lo4     : unsigned(3 downto 0);
89    begin
90      -- Por defecto: todo en blanco y DP apagados
91      d7 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
92      d6 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
93      d5 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
94      d4 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
95      d3 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
96      d2 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
97      d1 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
98      d0 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
99      dp_mask_8 <= (others => '1');
100
101     -- A y B siempre en decimal, 2 digitos cada uno
```

```
102     a_dec2 := u4_to_two_decimal_nibbles(
            operand_a_unsigned_4bit);
103     b_dec2 := u4_to_two_decimal_nibbles(
            operand_b_unsigned_4bit);
104
105     d7 <= unsigned_nibble_to_seven_segment_active_low(
            a_dec2(7 downto 4)); -- A decenas
106     d6 <= unsigned_nibble_to_seven_segment_active_low(
            a_dec2(3 downto 0)); -- A unidades
107
108     d5 <= unsigned_nibble_to_seven_segment_active_low(
            b_dec2(7 downto 4)); -- B decenas
109     d4 <= unsigned_nibble_to_seven_segment_active_low(
            b_dec2(3 downto 0)); -- B unidades
110
111     -- Zona de resultado (AN3..AN0)
112     if alu_result.flags.divide_by_zero_flag = '1' then
113       -- Error de division: poner 'E' en el digito mas a
              la derecha (AN0)
114       d3 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
115       d2 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
116       d1 <= SEVEN_SEGMENT_BLANK_ACTIVE_LOW;
117       d0 <= SEVEN_SEGMENT_CHAR_E_ACTIVE_LOW;
118       dp_mask_8(0) <= '0'; -- enciende DP0 para destacar
              el error
119     else
120       case current_operation is
121         when OPERATION_DIVIDE =>
122           -- Resultado en dos partes: [quot(2 dig)][rem
                (2 dig)] en AN3..AN0
123           q_dec2 := u4_to_two_decimal_nibbles(alu_result
                .quotient_4bit_unsigned);
124           m_dec2 := u4_to_two_decimal_nibbles(alu_result
                .remainder_4bit_unsigned);
125
126           d3 <=
                unsigned_nibble_to_seven_segment_active_low
                (q_dec2(7 downto 4)); -- Q decenas
127           d2 <=
                unsigned_nibble_to_seven_segment_active_low
                (q_dec2(3 downto 0)); -- Q unidades
128           d1 <=
                unsigned_nibble_to_seven_segment_active_low
```

```
            (m_dec2(7 downto 4)); -- R decenas
129         d0 <=
            unsigned_nibble_to_seven_segment_active_low
            (m_dec2(3 downto 0)); -- R unidades
130
131       when others =>
132         -- ADD/SUB/MUL: tomar result_8bit_unsigned
            (0..255) y mostrar 4 digitos decimales
133         r_dec4 := u8_to_four_decimal_nibbles(
            alu_result.result_8bit_unsigned);
134
135         d3 <=
            unsigned_nibble_to_seven_segment_active_low
            (r_dec4(15 downto 12)); -- miles (sera 0 en
             0..255)
136         d2 <=
            unsigned_nibble_to_seven_segment_active_low
            (r_dec4(11 downto 8));  -- centenas
137         d1 <=
            unsigned_nibble_to_seven_segment_active_low
            (r_dec4(7  downto 4));  -- decenas
138         d0 <=
            unsigned_nibble_to_seven_segment_active_low
            (r_dec4(3  downto 0));  -- unidades
139     end case;
140   end if;
141 end process;
142
143 -- LEDs de banderas
144 led_carry_flag_active_high          <= alu_result.
    flags.carry_flag;
145 led_borrow_flag_active_high         <= alu_result.
    flags.borrow_flag;
146 led_zero_flag_active_high           <= alu_result.
    flags.zero_flag;
147 led_overflow_flag_active_high       <= alu_result.
    flags.overflow_flag;
148 led_divide_by_zero_flag_active_high <= alu_result.
    flags.divide_by_zero_flag;
149
150 -- Instancia del mux de 8 digitos
151 sevenseg_driver_inst: entity work.sevenseg_mux
152   generic map(
```

```
153       CLOCK_FREQUENCY_HZ     => 100_000_000,
154       DIGIT_REFRESH_RATE_HZ => 1000
155     )
156     port map(
157       clk_system                          => clk_system,
158       rst_sync_active_high                =>
             rst_sync_active_high,
159       digit7_segment_pattern_active_low   => d7,
160       digit6_segment_pattern_active_low   => d6,
161       digit5_segment_pattern_active_low   => d5,
162       digit4_segment_pattern_active_low   => d4,
163       digit3_segment_pattern_active_low   => d3,
164       digit2_segment_pattern_active_low   => d2,
165       digit1_segment_pattern_active_low   => d1,
166       digit0_segment_pattern_active_low   => d0,
167       decimal_points_active_low_per_digit => dp_mask_8,
168       seg_active_low                      =>
             seg_active_low,
169       dp_active_low                       =>
             dp_active_low,
170       an_active_low                       =>
             an_active_low
171     );
172 end architecture;
```

Listing 5: Módulo TOP: integra ALU, formato de salida y driver de 7 segmentos.

Qué integra este *top* (y por qué está ordenado así). Piensa en `alu_top` como el "director de orquesta": convierte los **switches** en operandos, selecciona la operación, llama a las **funciones** del paquete de la ALU y, con ayuda de `sevenseg_pkg`, **formatea** todo para los 8 displays. La idea es ver al mismo tiempo: **A** (dos dígitos), **B** (dos dígitos) y el **resultado** (cuatro dígitos), sin ambigüedades.

Entradas y casting. Las señales a_switches_4bit_active_high, b_switches_4bit_active_high se "castean" a `unsigned` de 4 bits, tal como exige `numeric_std`. Esto evita mezclar `std_logic_vector` sin semántica aritmética con operaciones matemáticas.

Selección de operación. Un `with-select` mapea `operation_select_2bit` a `operation_t`. Es legible (00,01,10,11) y robusto: cualquier otro valor cae

en `DIVIDE` por la cláusula `others` (útil si los switches generan *glitches* al moverlos).

Camino de datos combinacional. El `process` central llama a la función correcta (`add/sub/mul/div`) y recibe un `alu_result_record_t` que ya trae **datos** y **banderas**. No hay registros aquí: es lógica combinacional pura (simple y didáctica).

Formateo a 7 segmentos (la "magia" visible). Sección clave:

- **AN7..AN6**: `A` en **decimal** (decenas y unidades).
- **AN5..AN4**: `B` en **decimal**.
- **AN3..AN0**: **resultado**.
  - **ADD/SUB/MUL**: `result_8bit_unsigned` convertido a 4 dígitos decimales (0..255 $\rightarrow$ 0000..0255).
  - **DIV**: concatenamos $[Q(2díg)][R(2díg)]$ para que *cociente y residuo* queden claritos a simple vista.
  - **Divide-by-zero**: "E" en AN0 y `DP0` encendido. `A` y `B` se mantienen visibles para entender el contexto del error.

Banderas a LEDs. Las cinco banderas (`carry, borrow, zero, overflow, divide_by_zero`) salen *as is* a los LEDs. Esto te permite "correlacionar" lo que ves en los displays con el estado interno de la operación.

Driver de 7 segmentos. Se instancia `sevenseg_mux` con 100 MHz y 1 kHz por dígito, lo que da $\sim$ 8 kHz de refresco total (suave para el ojo humano). El `dp_mask_8` deja todos los puntos decimales apagados salvo cuando quieres **marcar error** en división.

5. Asignación de pines (XDC)

Reloj y reset

Señal	Descripción	PIN	IOSTANDARD
`clk_system`	Reloj del sistema (p.ej. 100 MHz)	**E3**	LVCMOS33
`rst_sync_active_high`	Reset síncrono, activo en alto	**V10**	LVCMOS33

Entradas de datos (switches)

Puerto	Bit	PIN	IOSTANDARD
a_switches_4bit_active_high	[0]	**J15**	LVCMOS33
a_switches_4bit_active_high	[1]	**L16**	LVCMOS33
a_switches_4bit_active_high	[2]	**M13**	LVCMOS33
a_switches_4bit_active_high	[3]	**R15**	LVCMOS33
b_switches_4bit_active_high	[0]	**R17**	LVCMOS33
b_switches_4bit_active_high	[1]	**T18**	LVCMOS33
b_switches_4bit_active_high	[2]	**U18**	LVCMOS33
b_switches_4bit_active_high	[3]	**R13**	LVCMOS33

Selector de operación

Puerto	Bit	PIN	IOSTANDARD
operation_select_2bit	[0]	**T8**	LVCMOS33
operation_select_2bit	[1]	**U8**	LVCMOS33

Displays de 7 segmentos — segmentos y punto decimal

Puerto	Bit	PIN	IOSTANDARD
seg_active_low	[6]	**T10**	LVCMOS33
seg_active_low	[5]	**R10**	LVCMOS33
seg_active_low	[4]	**K16**	LVCMOS33
seg_active_low	[3]	**K13**	LVCMOS33
seg_active_low	[2]	**P15**	LVCMOS33
seg_active_low	[1]	**T11**	LVCMOS33
seg_active_low	[0]	**L18**	LVCMOS33
dp_active_low	—	**H15**	LVCMOS33

Displays de 7 segmentos — ánodos (activos en bajo)

Puerto	Índice	PIN	IOSTANDARD
an_active_low	[0]	**J17**	LVCMOS33
an_active_low	[1]	**J18**	LVCMOS33
an_active_low	[2]	**T9**	LVCMOS33
an_active_low	[3]	**J14**	LVCMOS33
an_active_low	[4]	**P14**	LVCMOS33
an_active_low	[5]	**T14**	LVCMOS33
an_active_low	[6]	**K2**	LVCMOS33
an_active_low	[7]	**U13**	LVCMOS33

LEDs de estado (flags)

Señal (LED)	Descripción	PIN	IOSTANDARD
led_borrow_flag_active_high	Borrow (resta)	**H17**	LVCMOS33
led_carry_flag_active_high	Carry (suma)	**K15**	LVCMOS33
led_divide_by_zero_flag_active_high	Divide-by-zero	**J13**	LVCMOS33
led_overflow_flag_active_high	Overflow	**N14**	LVCMOS33
led_zero_flag_active_high	Zero	**R18**	LVCMOS33

6. Resultados

Suma (ADD)

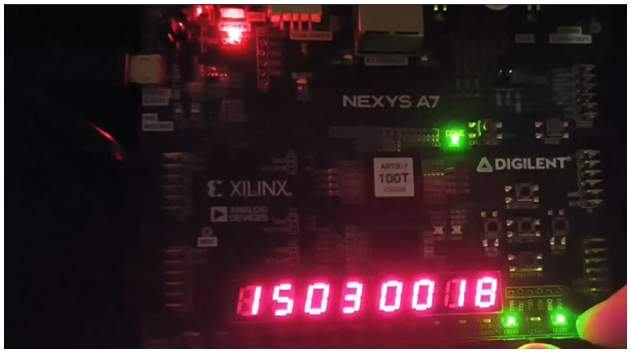


Figura 3: ADD con $A = 15$ y $B = 3$. Displays: A=15 (AN7..AN6), B=03 (AN5..AN4), Resultado=0018 (AN3..AN0).

Resta (SUBTRACT)

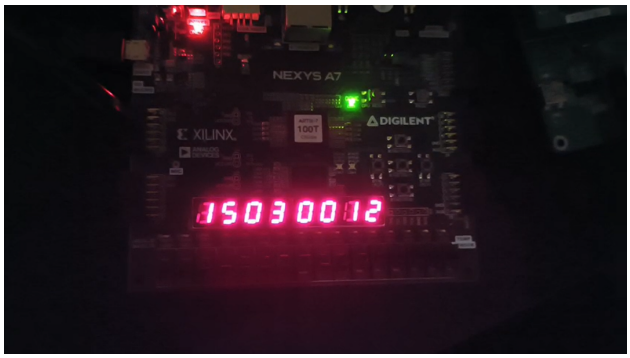


Figura 4: SUB con $A = 15$ y $B = 3$. Displays: A=15, B=03, Resultado=0012 (hex bajo) pero mostrado como decimal 0012

Multiplicación (MULTIPLY)

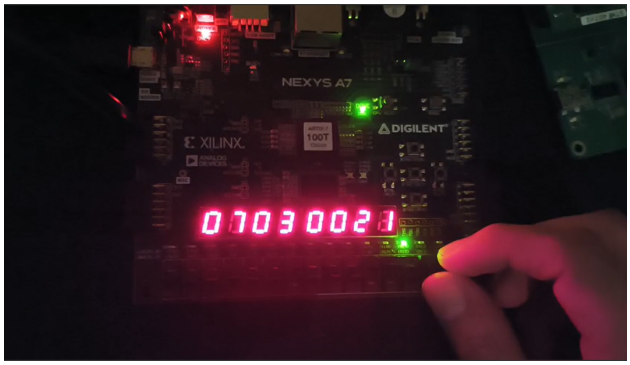


Figura 5: MUL con $A = 7$ y $B = 3$. Displays: A=07, B=03, Resultado=0021.

División (DIVIDE)

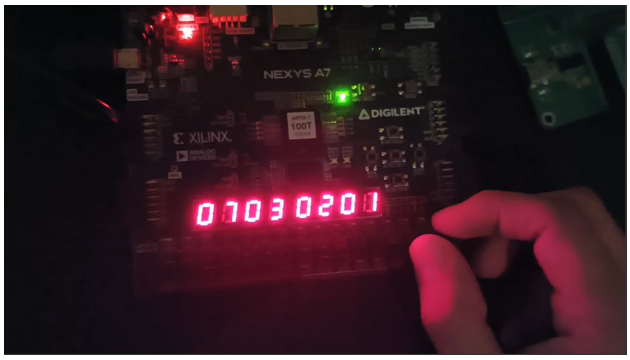


Figura 6: DIV con $A = 7$ y $B = 3$. Displays: A=`07`, B=`03`, Resultado=`[Q=02] [R=01]` en AN3..AN0.